

Core JavaScript Concepts

Q1: What is hoisting?

Explanation:

Hoisting is JavaScript's behavior of moving declarations (variables and functions) to the top of their scope before code execution. This means variables declared with `var` are accessible before their definition but will be `undefined`. However, variables declared with `let` and `const` exist in the **Temporal Dead Zone (TDZ)** until they are assigned a value.

```
console.log(x); // undefined (var is hoisted)
var x = 5;

console.log(y); // ReferenceError (let/const in TDZ)
let y = 10;
```

Q2: `==` vs `===`

Explanation:

- `==` (abstract equality) checks only values and allows **type coercion**.
- `===` (strict equality) checks both values and types.

```
0 == "0"    // true (coercion converts "0" to a number)
0 === "0"   // false (different types)
```

Q3: Explain closures

Explanation:

A closure is a function that remembers the variables from its **lexical scope** even when it is executed outside that scope. This allows data encapsulation and private variables.

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  };
}

const counter = outer();
counter(); // 1 (remembers 'count')
```

Q4: Event loop example

Explanation:

JavaScript has a non-blocking, asynchronous execution model. The **event loop** ensures that asynchronous callbacks (like `setTimeout` or Promises) are executed after synchronous code.

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
```

Output: `Start → End → Promise → Timeout`

Q5: `let` vs `const` vs `var`

Explanation:

- `var` : Function-scoped, can be redeclared, hoisted with `undefined`.
- `let` : Block-scoped, cannot be redeclared, hoisted in **TDZ**.
- `const` : Block-scoped, immutable reference.

```
if (true) {
  var a = 1; // function-scoped
  let b = 2; // block-scoped
  const c = 3; // block-scoped, immutable
}
```

```
console.log(a); // 1
console.log(b); // ReferenceError
```

Functions

Q6: Arrow vs regular functions

Explanation:

Arrow functions **do not have their own `this`**, they inherit `this` from their lexical scope.

```
const obj = {
  name: "Alice",
  regular: function() { console.log(this.name); }, // "Alice"
  arrow: () => console.log(this.name) // undefined (no 'this')
};
obj.regular();
obj.arrow();
```

Q7: Pure function example

Explanation:

A **pure function**:

- Returns the same output for the same input.
- Has no side effects (e.g., modifying external variables).

```
// Pure: Same input → same output, no side effects
function add(a, b) { return a + b; }

// Impure: Modifies external state
let total = 0;
function impureAdd(x) { total += x; }
```

Asynchronous JS

Q8: Callback → Promise → async/await

Explanation:

Handling asynchronous operations:

- **Callback Hell** (nested callbacks, hard to maintain)
- **Promise** (chaining, better structure)
- **Async/Await** (more readable, looks synchronous)

```
// Callback Hell
fetchData((data) => {
  process(data, (result) => {
    save(result, () => console.log("Done"));
  });
});
```

```
// Promise
fetchData()
  .then(process)
  .then(save)
  .catch(console.error);
```

```
// Async/Await
async function workflow() {
  try {
    const data = await fetchData();
    const result = await process(data);
    await save(result);
  } catch (err) { console.error(err); }
}
```

Q9: Promise.all

Explanation:

`Promise.all` runs multiple promises in parallel and waits for all to resolve. If any **fail**, the entire operation rejects.

```
const promises = [fetch(url1), fetch(url2)];
Promise.all(promises)
  .then(([res1, res2]) => console.log(res1, res2))
  .catch(err => console.log("One failed"));
```

Objects & Prototypes

Q10: `null` vs `undefined`

Explanation:

- `undefined` : A variable that has been declared but **not assigned a value**.
- `null` : An **explicit assignment** of "no value."

```
let a;           // undefined
let b = null;    // explicitly null
console.log(typeof a); // "undefined"
console.log(typeof b); // "object" (historical bug)
```

Q11: Prototypal inheritance

Explanation:

JavaScript uses **prototypal inheritance** where objects can inherit properties from other objects.

```
const parent = { greet() { return "Hello!" } };
const child = Object.create(parent);
console.log(child.greet()); // "Hello!" (inherited)
```

ES6+ Features

Q12: Template literals

Explanation:

Template literals allow embedding variables inside strings using backticks (`).

```
const name = "Bob";
console.log(`Hi, ${name}!`); // Hi, Bob!
```

Q13: Spread vs Rest

Explanation:

- **Spread** (`...`) expands elements of an array/object.
- **Rest** (`...`) collects multiple values into an array.

```
// Spread
const arr1 = [1, 2];
const arr2 = [...arr1, 3]; // [1, 2, 3]

// Rest
function sum(...nums) { return nums.reduce((a, b) => a + b); }
sum(1, 2, 3); // 6
```

Tricky Questions

Q14: `setTimeout` with 0 delay

Explanation:

Even with 0 delay, the callback runs **after synchronous code** due to the **event loop**.

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
```

```
console.log("End");  
// Output: Start → End → Timeout
```

Q15: `typeof []`

Explanation:

An array is an **object** in JavaScript.

```
console.log(typeof []); // "object"  
console.log(Array.isArray([])); // true
```

Additional Questions

Q16: What is the output?

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```

Output: 3, 3, 3 (var is function-scoped, shares the same `i`)

Fix: Use `let` (block-scoped) or an **IIFE**.

This document now contains **detailed explanations** for all questions!

Some more JS questions

JavaScript Interview Questions with Detailed Explanations

1. Difference Between `forEach` and `map`

Q: What's the difference between `forEach` and `map`?

A:

Both `forEach` and `map` iterate over arrays, but they have different use cases:

- `forEach` executes a function for each element in an array but does **not return** anything. It is mainly used for performing side effects like logging or modifying external variables.

```
const arr = [1, 2, 3];
arr.forEach(x => console.log(x * 2)); // Logs: 2, 4, 6
// Returns: undefined
```

- `map` returns a **new array** with transformed values, making it useful for creating modified versions of arrays.

```
const doubled = arr.map(x => x * 2); // [2, 4, 6]
```

2. `map`, `filter`, `reduce`

Q: Explain `map`, `filter`, and `reduce` with examples.

A:

- `map`: Used to transform each item in an array.

```
const nums = [1, 2, 3];
const squares = nums.map(x => x * x); // [1, 4, 9]
```

- `filter`: Used to create a new array containing only elements that meet a certain condition.

```
const evens = nums.filter(x => x % 2 === 0); // [2]
```

- `reduce`: Used to reduce an array into a single value (like sum, max, etc.).


```
const sum = nums.reduce((acc, x) => acc + x, 0); // 6
```

3. Closures

Q: How would you use a closure to create a private counter?

A:

Closures allow functions to "remember" variables even after they are executed. This enables data privacy.

```
function createCounter() {  
  let count = 0;  
  return {  
    increment() { count++; },  
    getCount() { return count; }  
  };  
}  
  
const counter = createCounter();  
counter.increment();  
console.log(counter.getCount()); // 1  
// `count` is inaccessible outside!
```

4. Currying

Q: What is currying? Give an example.

A:

Currying is a technique where a function that takes multiple arguments is transformed into a sequence of functions that take one argument at a time.

```
// Non-curried function  
const add = (a, b) => a + b;  
  
// Curried version
```

```
const curryAdd = a => b => a + b;
const add5 = curryAdd(5);
console.log(add5(3)); // 8
```

5. call, apply, and bind

Q: Explain call, apply, and bind.

A:

These methods control the `this` context in function calls.

- **call**: Invokes a function with a given `this` and individual arguments.

```
function greet(msg) { console.log(`${msg}, ${this.name}`); }
const user = { name: "Alice" };
greet.call(user, "Hello"); // "Hello, Alice"
```

- **apply**: Similar to **call** but takes arguments as an array.

```
greet.apply(user, ["Hi"]); // "Hi, Alice"
```

- **bind**: Returns a **new function** with a fixed `this` context.

```
const boundGreet = greet.bind(user);
boundGreet("Hey"); // "Hey, Alice"
```

6. Debouncing vs Throttling

Q: Explain debouncing and throttling with real-world examples.

A:

- **Debouncing**: Delays execution until after a pause in events (e.g., search input suggestions).

```
function debounce(fn, delay) {
  let timeout;
  return (...args) => {
    clearTimeout(timeout);
    timeout = setTimeout(() => fn(...args), delay);
  };
}

const searchInput = debounce(() => console.log("Searching..."), 500);
```

- **Throttling:** Ensures execution at most once per time interval (e.g., scroll events).

```
function throttle(fn, limit) {
  let lastRun;
  return (...args) => {
    if (!lastRun || Date.now() - lastRun >= limit) {
      fn(...args);
      lastRun = Date.now();
    }
  };
}

const handleScroll = throttle(() => console.log("Scrolled"), 1000);
```

7. Prototypal Inheritance

Q: How would you implement inheritance without **class**?

A:

By using **Object.create()** to inherit properties.

```
const animal = { eats: true };
const rabbit = Object.create(animal, { jumps: { value: true } });
console.log(rabbit.jumps); // true (own property)
console.log(rabbit.eats);  // true (inherited)
```

8. Event Delegation

Q: What is event delegation?

A:

Attaching a **single** event listener to a parent to handle events for multiple child elements.

```
document.getElementById("parent").addEventListener("click", (e) => {  
  if (e.target.matches("button.child")) {  
    console.log("Button clicked!");  
  }  
});
```

9. typeof vs instanceof

Q: What's the difference?

A:

- **typeof** checks primitive types but returns "object" for **null** and arrays.

```
console.log(typeof []); // "object"
```

- **instanceof** checks the prototype chain.

```
console.log([] instanceof Array); // true
```

10. setTimeout and setInterval Gotchas

Q: What's wrong with this code?

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```

A:

This prints 3, 3, 3 because `var` is function-scoped. To fix it, use `let` or an IIFE.

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```
